

Breaking VNC Clients with Evil Servers

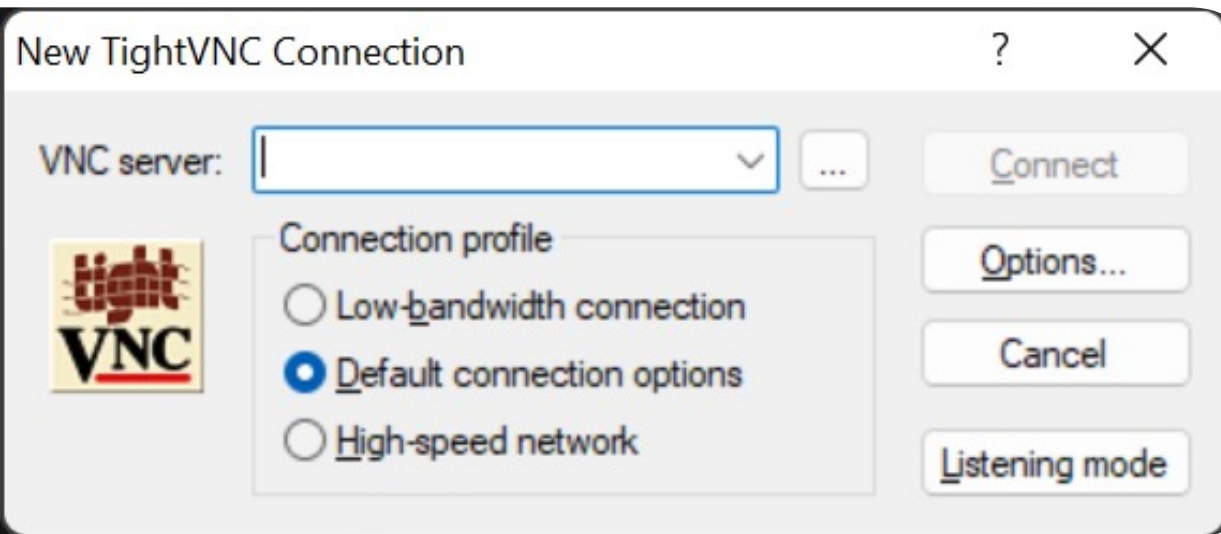
Eugene Lim
@spaceraccoonsec
spaceraccoon.dev





Disclaimer

This is actually a starter guide for fellow vulnerability research n00bs, not a deep dive into VNC



Sometimes you need
to connect to a
desktop remotely...

Virtual Network Computing (VNC) is just one of many remote desktop protocols.

VNC

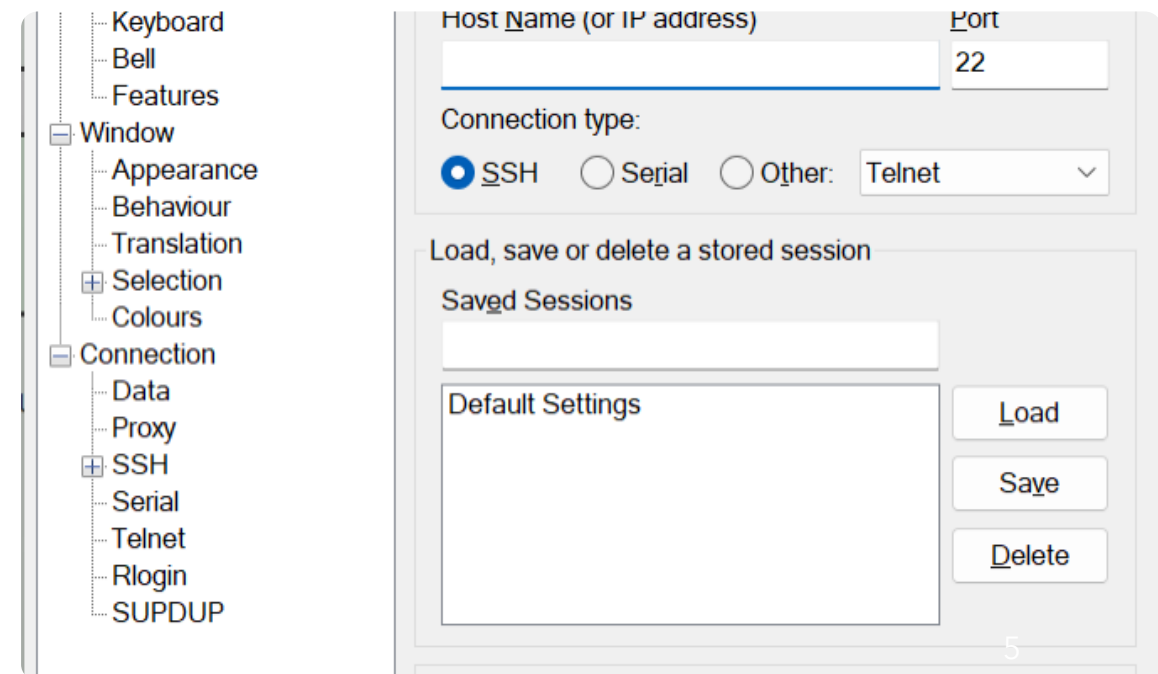
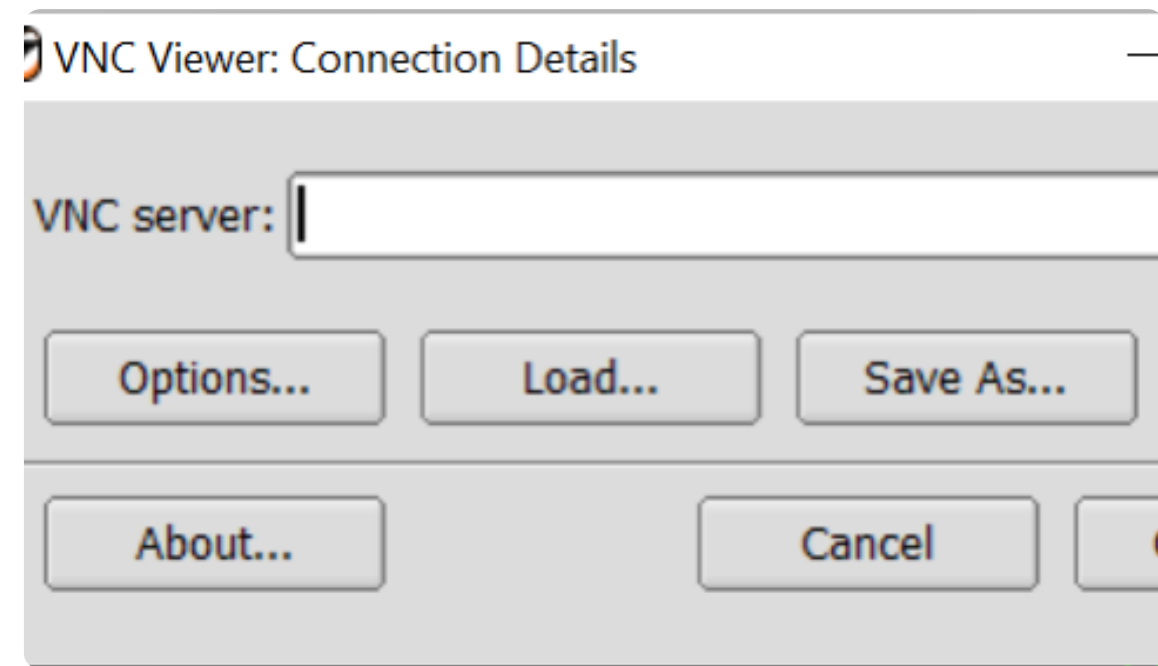
RDP

ESXi

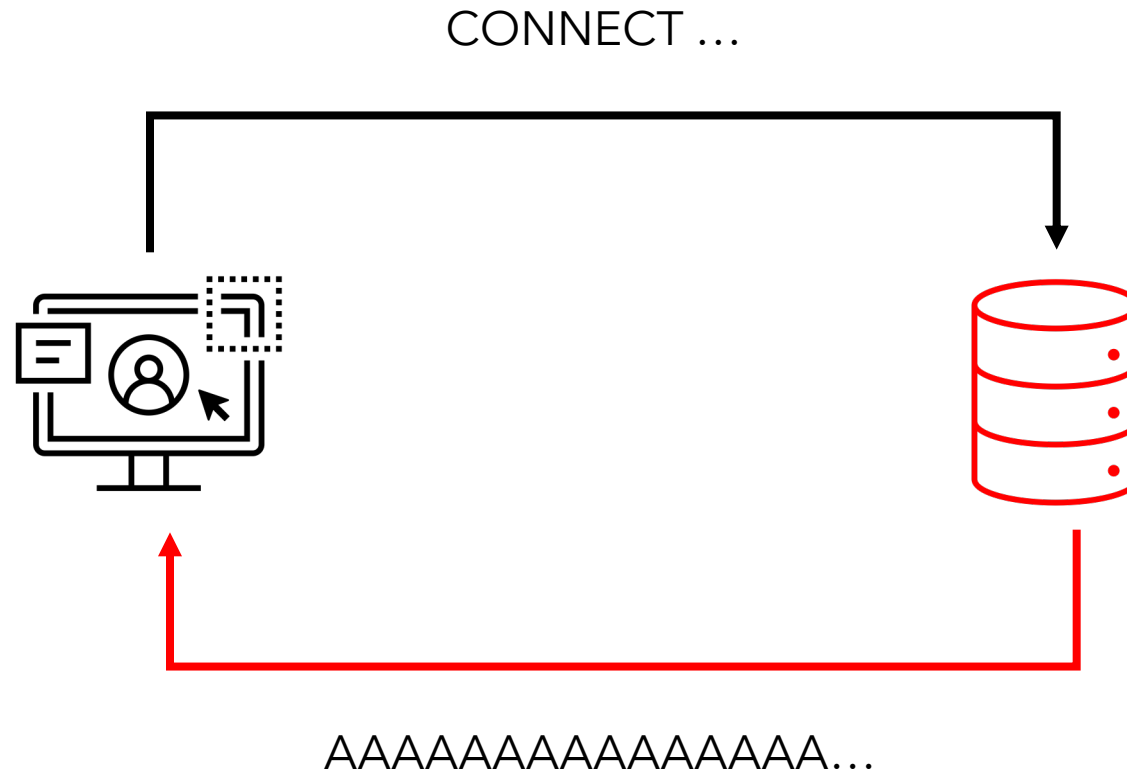
Citrix

SPICE

Practicing vulnerability research and code review on open-source remote desktop clients.



Why clients? It's an interesting attack vector.



I created a
simple plan
of attack.



Read specification



Source code review



Write evil server



Run test cases



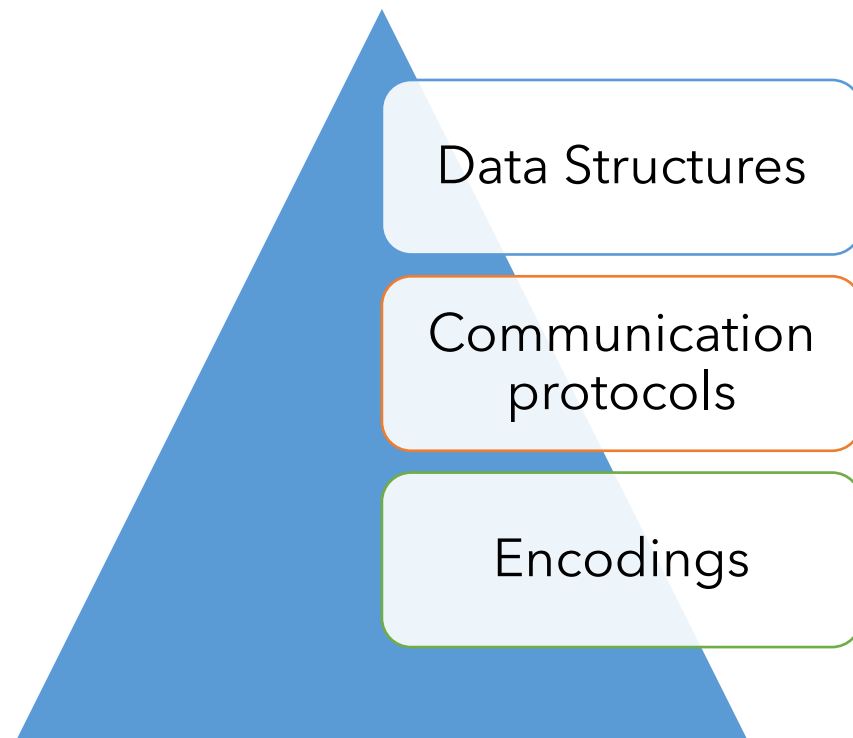
???

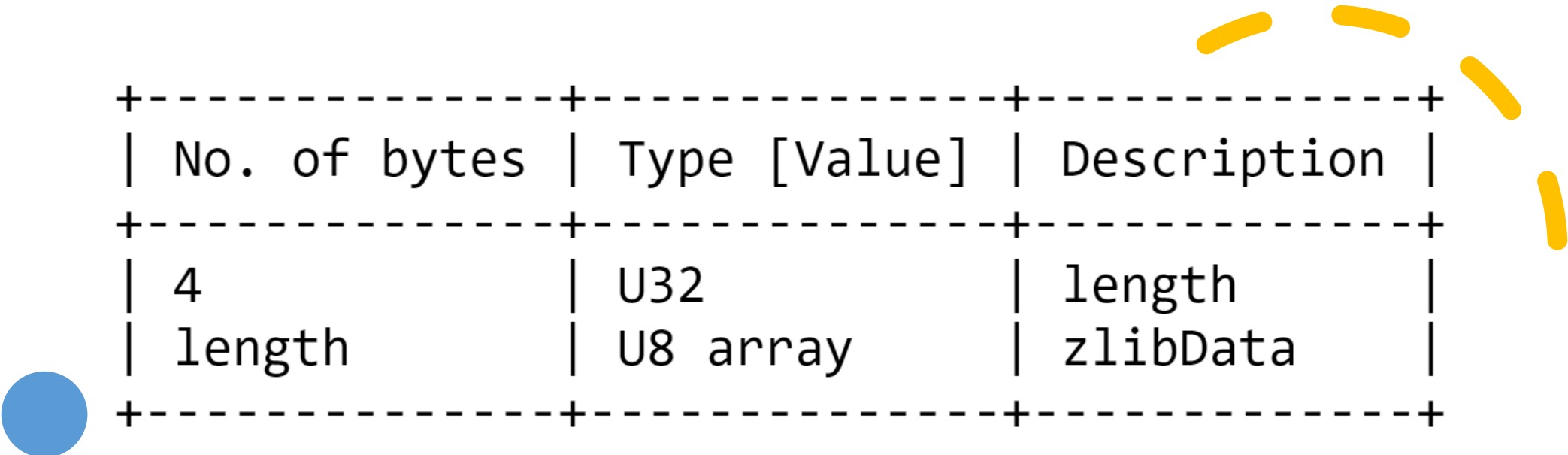


Profit

7.3.	Initialization Messages
7.3.1.	ClientInit
7.3.2.	ServerInit
7.4.	Pixel Format Data Structure
7.5.	Client-to-Server Messages
7.5.1.	SetPixelFormat
7.5.2.	SetEncodings
7.5.3.	FramebufferUpdateRequest
7.5.4.	KeyEvent
7.5.5.	PointerEvent
7.5.6.	ClientCutText
7.6.	Server-to-Client Messages
7.6.1.	FramebufferUpdate
7.6.2.	SetColorMapEntries
7.6.3.	Bell
7.6.4.	ServerCutText
7.7.	Encodings
7.7.1.	Raw Encoding
7.7.2.	CopyRect Encoding
7.7.3.	RRE Encoding
7.7.4.	Hextile Encoding
7.7.5.	TRLE
7.7.6.	ZRLE
7.8.	Pseudo-Encodings
7.8.1.	Cursor Pseudo-Encoding
7.8.2.	DesktopSize Pseudo-Encoding

RFC6143 “The Remote Framebuffer Protocol” describes a short and simple protocol (39 pages).





No. of bytes	Type [Value]	Description
4	U32	length
length	U8 array	zlibData

Tag-Length-Value (TLV) schemes are a great source of buffer overflow bugs.

After generating possible exploit scenarios,
run a code review sieve on multiple targets.



var	location
gp_offset	file:///0:0:0:0
fp_offset	file:///0:0:0:0
overflow_arg_area	file:///0:0:0:0
reg_save_area	file:///0:0:0:0
argc	file:///home/cyber/code.c:1:14:1:17
argv	file:///home/cyber/code.c:1:27:1:30
size1	file:///home/cyber/code.c:3:9:3:13
size2	file:///home/cyber/code.c:4:9:4:13
dst	file:///home/cyber/code.c:6:11:6:13
src	file:///home/cyber/code.c:7:11:7:13

The VS Code plugin
makes CodeQL
more user-friendly.

Craft a custom source-sink CodeQL query and step through the data flows.

```
import cpp
import semmle.code.cpp.dataflow.TaintTracking

class StreamToMemcpyConfiguration extends DataFlow::Configuration {
  StreamToMemcpyConfiguration() { this = "StreamToMemcpyConfiguration" }

  override predicate isSource(DataFlow::Node source) {
    exists (Function recv |
      source.asExpr().(FunctionCall).getTarget() = recv and
      (
        recv.hasQualifiedName("rdp_recv") or
        recv.hasQualifiedName("tcp_recv") or
        recv.hasQualifiedName("iso_recv")
      )
    )
  }

  override predicate isSink(DataFlow::Node sink) {
    exists (FunctionCall fc |
      sink.asExpr() = fc.getArgument(1) and
      fc.getTarget().hasQualifiedName("memcpy")
    )
  }
}

from Expr rdpRecv, Expr memcpy, StreamToMemcpyConfiguration config
where config.hasFlow(DataFlow::exprNode(rdpRecv), DataFlow::exprNode(memcpy))
select memcpy, "This 'memcpy' uses data from @$ as src.",
      rdpRecv, "call to 'rdp_recv'"
```

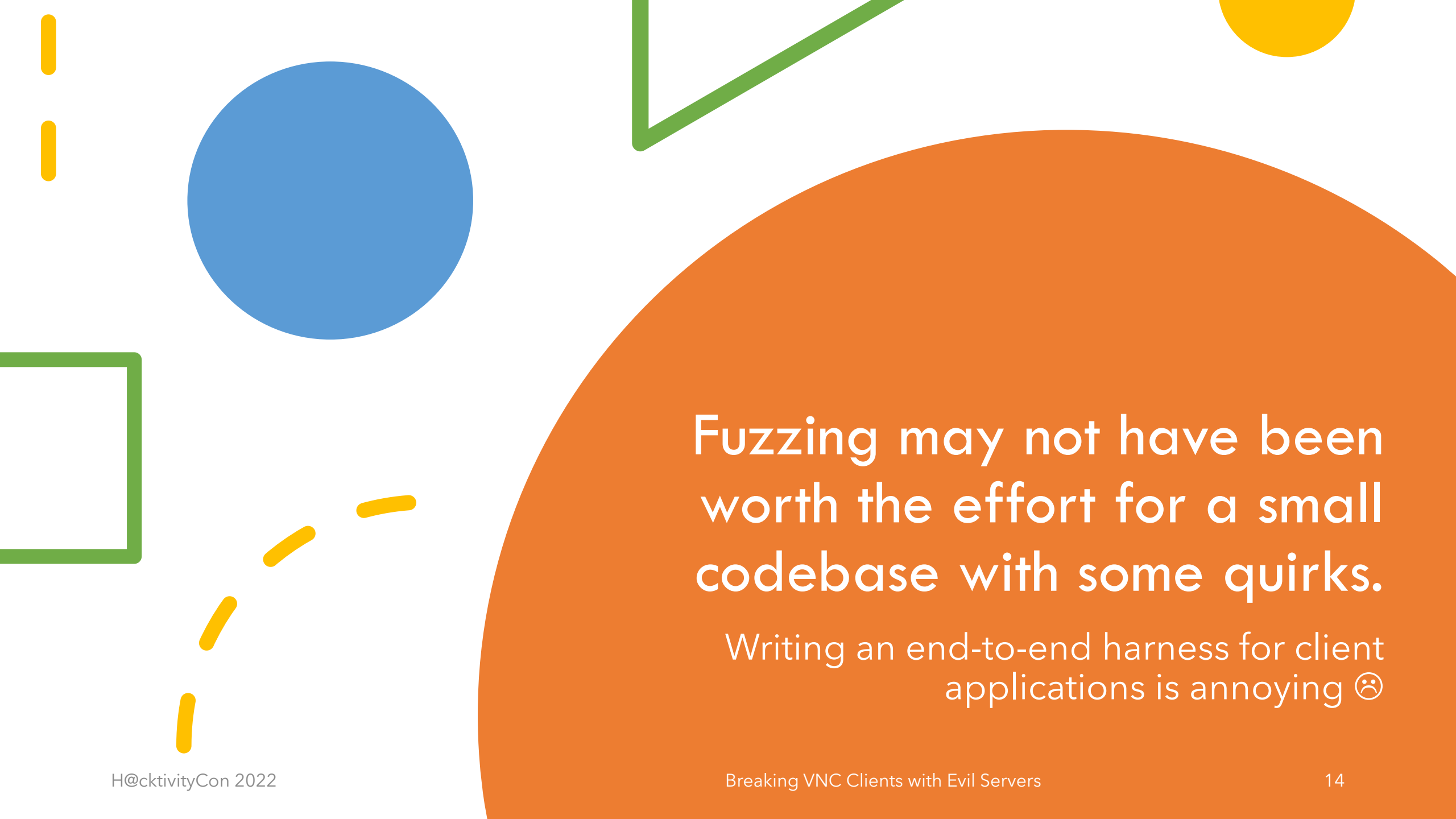
```

296 Palette palette;
297 readPalette(input, paletteSize, &palette);
298
299 for (size_t indexPixel = 0; indexPixel < tileLength;) {
300     UINT8 color = input->readUInt8();
301
302     size_t runLength = 1;
303     if (color >= 128) {
304         color -= 128;
305         runLength = readRunLength(input);
306     }
307
308     char * pixelsPtr = &pixels[indexPixel * m_bytesPerPixel];
309     for(size_t i = 0; i < runLength; i++) {
310         memcpy(pixelsPtr, &palette[color], m_bytesPerPixel);
311         pixelsPtr += m_bytesPerPixel;
312     }
313     indexPixel += runLength;
314 }
315 }
316
317 void ZrleDecoder::drawTile(FrameBuffer *fb,
318                             const Rect *tileRect,
319                             const vector<char> *pixels)
320 {
321     int width = tileRect->getWidth();
322     size_t fbBytesPerPixel = m_bytesPerPixel;
323
324     if (fbBytesPerPixel == 3) {

```

CodeQL identifies problem areas but don't avoid manual code review

- bufferSize argument in memcpy ✓
- runLength number of loops ✗
- Eventually overflows buffer and overwrites EIP
- Not detected by standard CodeQL rules



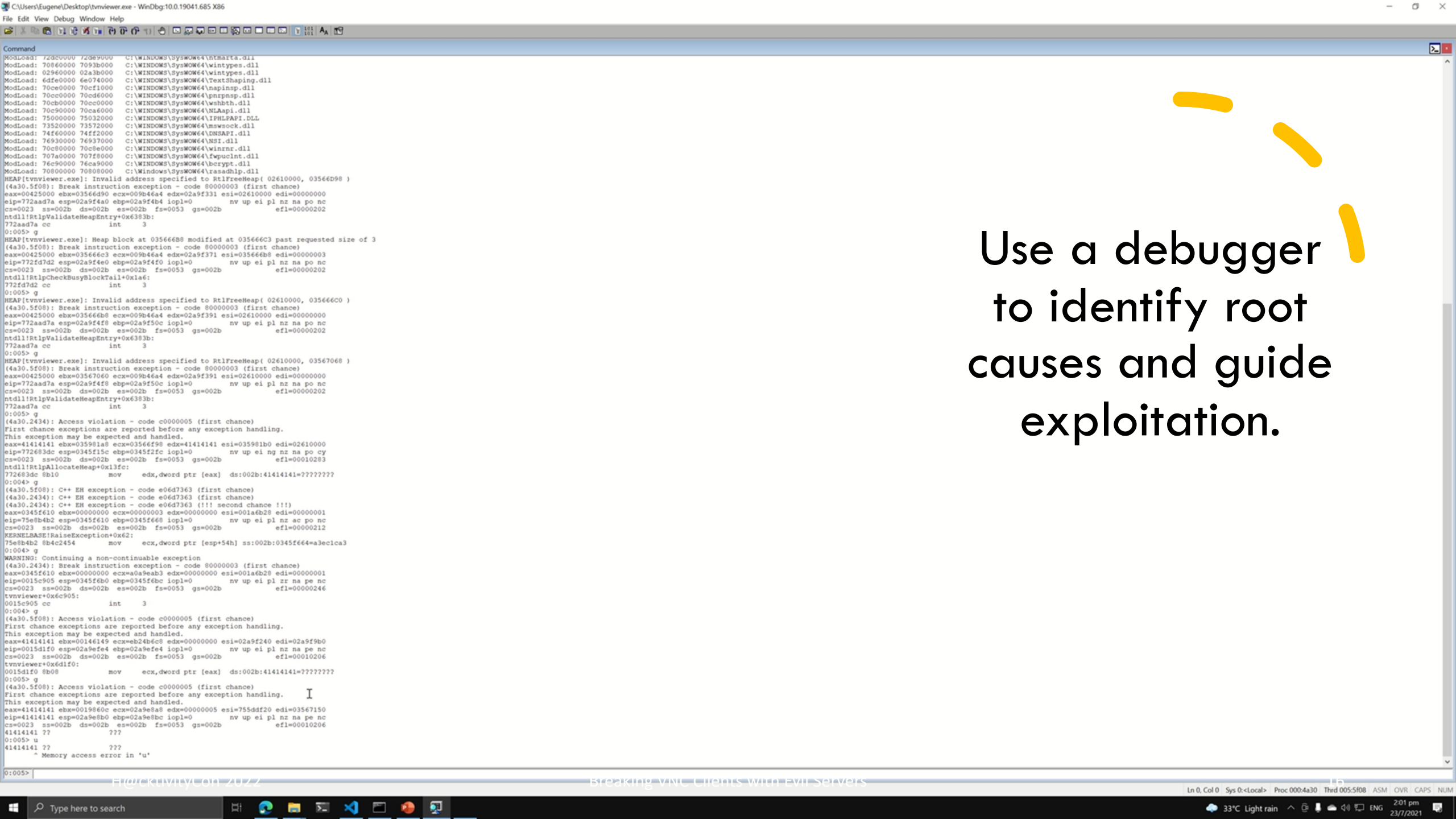
Fuzzing may not have been
worth the effort for a small
codebase with some quirks.

Writing an end-to-end harness for client
applications is annoying 😞

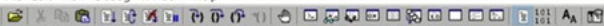
 evilvnc.py

 evilvnc.py

Refer to open-source implementations to build a simple evil VNC server.



Use a debugger
to identify root
causes and guide
exploitation.



Command

CommandLine: C:\Users\Eugene\Desktop\tnvviewer.exe

***** Path validation summary *****

Response	Time (ms)	Location
Deferred		srv*

Symbol search path is: srv*

Executable search path is:

ModLoad: 000f0000 001bc000 tnvviewer.exe

ModLoad: 77220000 773c3000 ntdll.dll

ModLoad: 755c0000 756b0000 C:\WINDOWS\SysWOW64\KERNEL32.DLL

ModLoad: 75d60000 75f74000 C:\WINDOWS\SysWOW64\KERNELBASE.dll

ModLoad: 76d50000 760b3000 C:\WINDOWS\SysWOW64\WS2_32.dll

ModLoad: 76b40000 76bfb000 C:\WINDOWS\SysWOW64\RPCRT4.dll

ModLoad: 6b2d0000 6b4e0000 C:\WINDOWS\WinSxS\x86_microsoft.windows.common-controls_6595b64144ccf1df_6.0.19041.1110_none_a8625c1886757984\COMCTL32.dll

ModLoad: 769f0000 76aaf000 C:\WINDOWS\SysWOW64\msvrt.dll

ModLoad: 75870000 75a06000 C:\WINDOWS\SysWOW64\USER32.dll

ModLoad: 75a10000 75a28000 C:\WINDOWS\SysWOW64\win32u.dll

ModLoad: 750d0000 750f3000 C:\WINDOWS\SysWOW64\GDI32.dll

ModLoad: 756b0000 7578c000 C:\WINDOWS\SysWOW64\gdi32full.dll

ModLoad: 76940000 769ef000 C:\WINDOWS\SysWOW64\COMDLG32.dll

ModLoad: 75b40000 75bbb000 C:\WINDOWS\SysWOW64\msvcp_win.dll

ModLoad: 75240000 754c1000 C:\WINDOWS\SysWOW64\cobase.dll

ModLoad: 75bc0000 75ce0000 C:\WINDOWS\SysWOW64\ucrtbase.dll

ModLoad: 00cb0000 004d0000 C:\WINDOWS\SysWOW64\ucrtbase.dll

ModLoad: 76c00000 76c87000 C:\WINDOWS\SysWOW64\shcore.dll

ModLoad: 761b0000 761f5000 C:\WINDOWS\SysWOW64\SHLWAPI.dll

ModLoad: 76370000 76923000 C:\WINDOWS\SysWOW64\SHELL32.dll

ModLoad: 76ab0000 76b2a000 C:\WINDOWS\SysWOW64\ADVAPI32.dll

ModLoad: 75790000 75805000 C:\WINDOWS\SysWOW64\sechost.dll

ModLoad: 73dc0000 73dc8000 C:\WINDOWS\SysWOW64\VERSION.dll

(4a30.6394): Break instruction exception - code 80000003 (first chance)

eax=00000000 ebx=00000000 ecx=9b200000 edx=00000000 esi=77232044 edi=7723260c

eip=772d1b72 esp=003ef73c ebp=003ef768 iopl=0

cs=0023 ss=002b ds=002b es=002b fs=0053 gs=002b eip=00000246

ntdll!LdrpDoDebuggerBreak+0x2b: int 3

772d1b72 cc

0:000>

Disclosure itself is an art.

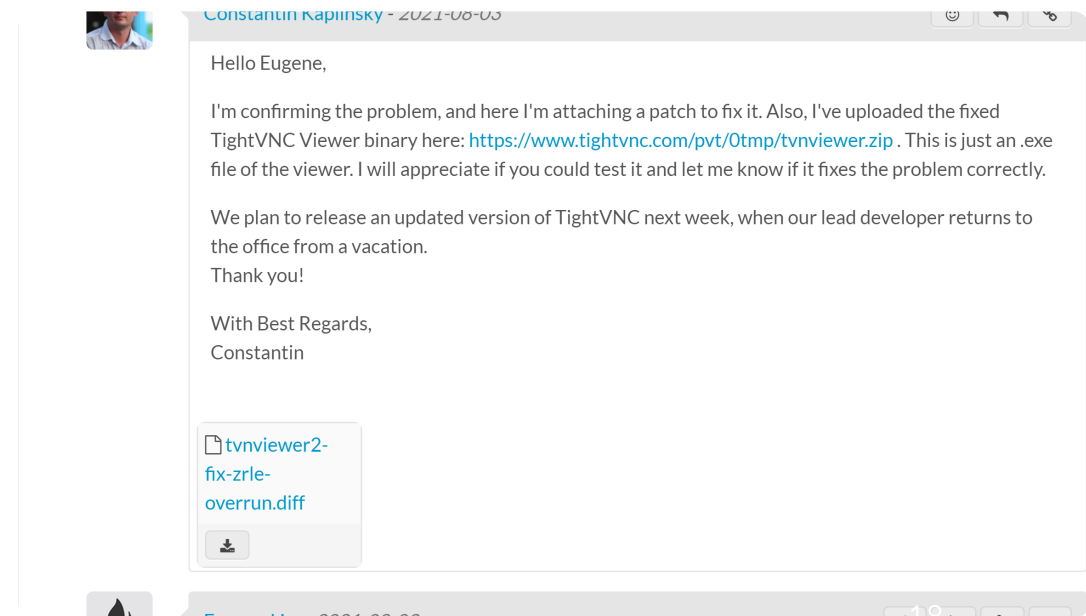
TightVNC: CVE-2021-42785

LibVNC: Bug clash with CrowdStrike two
weeks earlier 🙄

> Ah wow, looks like I was testing only on the 0.9.13 commits - could I check
> if this was patched because of a vulnerability that was already reported?

It was, by the guys at CrowdStrike.

Best,





Turns out, the real trick is to
read a specification, and ask
yourself: “If I implemented this
spec, what mistakes might I
make?”

@retr0id
“BGGP3: Chipping Out”

This workflow
provides a
good starting
point for
beginners.



Read specification



Source code review



Write evil server



Run test cases



??? (replace with fuzzing)



Profit

Thank you

Eugene Lim
@spaceraccoonsec
spaceraccoon.dev

